Introduction to

Python Programming

& Its Core Objects

Table of Contents

Chapter 1

Introduction to Python

Welcome to the world of Python programming! This chapter introduces you to one of the most widely used programming languages in the world. By the end of this chapter, you will understand what Python is, where it came from, why it is so popular, and how it is used across different industries. We will also write your very first Python program.

1.1 What is Python?

Python is a

Python	A high-level, general-purpose programming language that is designed to be easy to read and write. It is used to give instructions to a computer in a format that is much closer to plain English than most other languages.

Think of Python as a translator between you and the computer. You write your instructions in Python, and the computer understands and executes them.

🔎 Analogy — Instructions to a Computer

Imagine you want to tell a friend to make a cup of tea. You would say:

"Boil water. Add a tea bag. Pour water into the cup. Wait 3 minutes. Remove the bag."

Python works the same way — you give step-by-step instructions. The computer follows every step exactly as written. Python simply makes sure those steps are written in a way that the computer can understand.

Python is called a

High-Level Language: This means it is designed for humans, not machines. You write code that makes sense to you, and Python translates it into something the computer can process.

Interpreted Language: Python reads and executes your code line by line, one instruction at a time, rather than converting everything at once before running.

General-Purpose Language: You can use Python for a wide variety of tasks — from building websites to analyzing data to controlling robots.

1.2 History of Python and Guido van Rossum

Python did not appear suddenly. It was carefully designed and developed over several years. Understanding its history helps you appreciate why it is built the way it is.

The Creator — Guido van Rossum

Python was created by a Dutch programmer named Guido van Rossum. In the late 1980s, Guido was working at a research institute in the Netherlands called Centrum Wiskunde & Informatica (CWI). He found the programming tools available at the time too complex and not user-friendly.

He wanted to create a language that was simple, readable, and powerful enough for professional use. He began working on Python in December 1989, during his Christmas holidays.

Year	Milestone
1989	Guido van Rossum begins writing Python during the Christmas break at CWI, Netherlands.
1991	Python 0.9.0 is released publicly for the first time. It includes basic features like functions, exception handling, and core data types.
1994	Python 1.0 is officially released, introducing features such as lambda functions and the map/filter tools.
2000	Python 2.0 is released, bringing list comprehensions and improved memory management. This became widely adopted by developers worldwide.
2008	Python 3.0 is released — a major upgrade that fixed many design issues but was intentionally not backward compatible with Python 2.
2020	Python 2 reaches its official end of life. Python 3 becomes the only actively supported version.
Today	Python consistently ranks as one of the top 3 most popular programming languages worldwide.

Note

The name "Python" was not inspired by the snake. Guido van Rossum named it after a British comedy television show called Monty Python's Flying Circus, which he enjoyed. That is why Python's official documentation and tutorials often include playful references to the show.

From Student Project to Global Standard

What started as a personal project by one developer has grown into a globally recognized language used by millions of developers, data scientists, researchers, students, and businesses. Python is now maintained by the Python Software Foundation (PSF), a non-profit organization that ensures the language continues to evolve.

1.3 Why Python is Popular

Python's popularity did not happen by chance. There are very specific, practical reasons why it is chosen by beginners and industry experts alike.

1. Readable and Clean Syntax

Python code reads almost like a sentence in English. This makes it much easier to write, read, and understand — especially for beginners.

Compare the following task: Print "Hello, World!" on the screen.

Language	Code to Print Hello, World!
Java	public class Main { public static void main(String[] args) { System.out.println("Hello, World!"); } }
C++	#include <iostream> int main() { std::cout << "Hello, World!"; return 0; }
Python	print("Hello, World!")

As you can see, Python achieves the same result with far less code and no complicated setup. This simplicity is one of its greatest strengths.

2. Large and Supportive Community

Python has one of the largest developer communities in the world. This means:

- Thousands of free tutorials, videos, and books are available.
- If you face a problem, someone has likely already solved it — and shared the solution online.
- Active forums like Stack Overflow and the official Python Forum are filled with helpful members.

3. Massive Collection of Libraries

A **library** in programming is a collection of pre-written code that you can use in your own projects without writing it from scratch. Python has thousands of libraries for every purpose:

Purpose	Python Library
Data Analysis	pandas, NumPy
Machine Learning	scikit-learn, TensorFlow, PyTorch
Web Development	Django, Flask, FastAPI
Data Visualization	Matplotlib, Seaborn, Plotly
Automation / Scripting	os, shutil, subprocess

Working with APIs	requests, httpx

4. Versatile — Works Everywhere

Python works on all major operating systems — Windows, macOS, and Linux. You can write the same code on one platform and it will generally work on another without major changes.

5. Free and Open Source

Python is completely free to download and use. Its source code is publicly available, which means anyone can contribute to improving it. There is no licensing fee — making it accessible to students and organizations worldwide.

> 💡 **Pro Tip**
>
> When you search for Python help online, make sure you are looking at Python 3 resources. Python 2 is outdated and no longer supported. Look for dates — articles from 2019 onwards are more likely to use Python 3.

1.4 Applications of Python

One of the reasons Python is so valuable is that it is used in nearly every field of technology and science. Below are the major areas where Python is applied professionally.

Data Science and Analytics

Data Science involves extracting useful insights from large amounts of data. Organizations use data science to make informed business decisions, predict trends, and improve products.

Python is the **primary language** of data science. Libraries like **pandas** allow analysts to clean and organize data. **NumPy** provides powerful mathematical tools. **Matplotlib** and **Seaborn** allow the creation of professional charts and graphs.

Example use case: A retail company uses Python to analyze customer purchasing patterns and determine which products to stock more of before a festive season.

Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) refers to building systems that can think, learn, and make decisions. Machine Learning (ML) is a subset of AI where computers learn patterns from data rather than being explicitly programmed.

Python is the dominant language in AI and ML due to libraries like:

- **TensorFlow** — Developed by Google, used to build deep learning models.
- **PyTorch** — Developed by Meta, widely used in academic research and production AI systems.

- **scikit-learn** — Used for classical ML tasks like classification, regression, and clustering.

Example use case: A hospital uses Python-based ML models to detect patterns in medical scan images and flag potential cases for a doctor's review.

Web Development

Python is used to build the backend of websites and web applications — the part that runs on a server and handles data, logins, and business logic.

Popular Python web frameworks:

- **Django** — A full-featured framework used to build large-scale web applications. It includes everything — a database layer, user authentication, admin panels, and more.
- **Flask** — A lightweight framework ideal for smaller applications or APIs.
- **FastAPI** — A modern framework designed for building high-performance APIs.

Example use case: Instagram's backend was built using Django when the platform was first launched.

Automation and Scripting

Python is widely used to automate repetitive tasks, saving time and reducing human error. This is sometimes called scripting.

Common automation use cases:

- Automatically organizing files and folders on a computer.
- Sending scheduled emails or notifications.
- Filling out web forms or extracting data from websites (web scraping).
- Generating daily reports from databases.

Example use case: An accounts department writes a Python script that automatically downloads bank statements every morning, processes them, and generates an expense report — a task that previously took 2 hours manually.

📌 Python in the Real World

Here are some well-known companies that use Python as a core part of their technology:

- Google — Uses Python extensively in its search engine and internal tools.
- Netflix — Uses Python for its recommendation engine and content delivery systems.
- NASA — Uses Python for scientific simulations and spacecraft data analysis.
- Dropbox — The entire desktop client was originally written in Python.
- Spotify — Uses Python for backend data analysis and playlist recommendations.

1.5 Python Compared to Other Languages

It helps to understand how Python compares to other commonly used programming languages. This is not about which language is "better" — each has its purpose. The goal is to understand where Python excels and when another language might be chosen.

Feature	Python	Java	C++	JavaScript
Ease of Learning	Very easy — beginner-friendly	Moderate — more syntax rules	Difficult — complex memory management	Moderate — runs in browsers
Code Readability	Excellent — close to plain English	Moderate — more verbose	Low to moderate — low-level control	Moderate
Speed of Execution	Moderate — suitable for most tasks	Fast — compiled to bytecode	Very fast — low-level language	Fast for web apps
Primary Use	Data science, AI, automation, web	Enterprise software, Android apps	System software, game engines	Web frontend, Node.js backend
Community & Libraries	Very large ecosystem	Large, mature ecosystem	Large but more technical	Very large — npm ecosystem
Typical Syntax	print("Hello")	System.out.println("Hello");	cout << "Hello";	console.log("Hello");

📝 Note

Learning Python first is a strategic choice. Once you understand Python, learning other languages becomes significantly easier. The fundamental concepts — variables, loops, functions, conditions — are shared across all programming languages.

Chapter 2

Getting Started with Python

Now that you understand what Python is and why it matters, it is time to start using it. In this chapter, you will learn how to set up your Python environment, write your first program, and understand the fundamental rules that govern how Python code is written.

2.1 Writing Your First Program — Hello, World!

Every programmer's journey begins with a simple program: printing the text "Hello, World!" on the screen. This tradition goes back decades and serves as the quickest way to verify that your programming environment is set up correctly.

Step 1: Install Python

Before writing any code, you need to install Python on your computer. Python can be downloaded for free from the official website.

Once downloaded, run the installer. On Windows, ensure you check the box that says "Add Python to PATH" before clicking Install. This is a critical step that allows you to run Python from your system's terminal.Step 2: Open a Python Environment

Once Python is installed, you have two easy ways to start writing code:

- **The Python IDLE Editor** — A simple built-in editor that comes with Python. Good for beginners.
- **Visual Studio Code (VS Code)** — A professional code editor that is free, widely used, and highly recommended for this course.

Step 3: Write the Hello World Program

Open IDLE or VS Code. Create a new file. Type the following code exactly as shown:

```
# My first Python program
print("Hello, World!")
```

Figure 2.1 — Your first Python program

Now save the file with a .py extension, for example: **hello.py**. Then run the program.

In IDLE: Press F5 or go to Run → Run Module.

In VS Code: Click the Run button (▶) at the top right, or open the terminal and type: `python hello.py`

You will see the following output on your screen:

Output

```
Hello, World!
```

Congratulations! You have just written and run your first Python program.

2.2 Python Syntax Basics

Every language has a set of rules that define how it must be written. In programming, these rules are called **syntax**. Just as English has grammar rules, Python has its own syntax rules that must be followed for a program to work correctly.

Syntax	The set of rules that define the correct structure of statements in a programming language. Violating syntax rules causes an error, and the program will not run.

Here are the core syntax principles you will use throughout your Python journey:

Python is Case-Sensitive

Python treats uppercase and lowercase letters as different. This is an important rule that beginners often overlook.

```
# These are all different in Python:
name = "Alice"
Name = "Bob"
NAME = "Charlie"

# print and Print are different:
print("Hello")    # This works
Print("Hello")    # This causes an error — Python does not recognize Print
```

Figure 2.2 — Case sensitivity in Python

Statements End with a New Line

In Python, each instruction occupies its own line. You do not need a semicolon (;) at the end of each line, unlike languages such as Java or C++.

```
# Each of these is a separate statement on its own line:
name = "Alice"
age = 25
city = "Ahmedabad"

print(name)
```

```
print(age)
print(city)
```

Figure 2.3 — Statements in Python, one per line

Using print() to Display Output

The print() function is your main tool for displaying results. You place whatever you want to display inside the parentheses.

```
# Printing text (always use quotes around text):
print("Python is easy to learn")

# Printing numbers (no quotes needed):
print(2025)
print(3.14)

# Printing a calculation:
print(10 + 5)

# Printing multiple values:
print("My name is", "Alice", "and I am", 25, "years old")
```

Figure 2.4 — Different uses of the print() function

Output

```
Python is easy to learn
2025
3.14
15
My name is Alice and I am 25 years old
```

2.3 Comments in Python

As programs grow larger and more complex, it becomes important to explain what different sections of code do — both for yourself when you revisit the code later, and for others who may read it.

Comment	A line in your code that Python completely ignores when running the program. Comments are written for humans — not for the computer. They explain what the code does.

Single-Line Comments

In Python, a single-line comment begins with the hash symbol (#). Everything after # on that line is treated as a comment and is not executed.

```
# This is a single-line comment

name = "Alice"    # This stores the name Alice in a variable called name
age = 25          # This stores the number 25 in a variable called age

print(name)       # This prints the value of name to the screen
```

Figure 2.5 — Single-line comments in Python

Multi-Line Comments

If you want to write a longer explanation that spans multiple lines, you can either use multiple single-line comments or use a triple-quoted string.

```
# Option 1: Multiple single-line comments
# This program calculates the total cost of items
# after applying a discount percentage.
# Written by: Student Name | Date: 2025

# Option 2: Triple-quoted string (used as a multi-line comment)
"""
This program calculates the total cost of items
after applying a discount percentage.
Written by: Student Name | Date: 2025
"""
```

Figure 2.6 — Multi-line comments in Python

💡 Pro Tip

Develop the habit of writing comments from day one. Even simple programs benefit from a comment at the top explaining what they do. Future you — and any collaborator — will be grateful.

📌 Best Practices for Comments

- Write comments that explain WHY something is done, not just WHAT.

  Bad comment: `x = x + 1  # Add 1 to x`

  Good comment: `x = x + 1  # Increment counter for each student processed`

- Keep comments concise and accurate. An outdated comment is worse than no comment.
- Comment complex logic, edge cases, or non-obvious decisions.

2.4 Indentation Rules in Python

Indentation is one of Python's most distinctive and important features. While other programming languages use curly braces { } to group blocks of code, Python uses indentation — the spaces at the beginning of a line.

Indentation	The spaces or tab characters placed at the beginning of a line of code to indicate that it belongs to a specific code block (such as inside an if statement or a loop).

In Python, indentation is not optional — it is mandatory. Incorrect indentation causes an IndentationError and your program will not run.

Standard Indentation: 4 Spaces

The standard convention in Python is to use 4 spaces for each level of indentation. Most code editors (including VS Code and IDLE) automatically insert 4 spaces when you press the Tab key inside a Python file.

```
# Correct indentation:
temperature = 38

if temperature > 37:
    print("You have a fever.")          # 4 spaces — inside the if block
    print("Please rest and drink water.")  # Also inside the if block

print("Check complete.")               # 0 spaces — outside the if block
```

Figure 2.7 — Correct indentation example

Output

```
You have a fever.
Please rest and drink water.
Check complete.
```

Now look at what happens when indentation is incorrect:

```
# Incorrect indentation — this will cause an error:
temperature = 38

if temperature > 37:
print("You have a fever.")    # ERROR: Expected an indented block
```

Figure 2.8 — Indentation error example

Error Output

```
IndentationError: expected an indented block after 'if' statement on line 4
```

Indentation Levels

Each time you enter a new code block (inside an if, loop, or function), you add one more level of indentation. When the block ends, you return to the previous level.

```
# Two levels of indentation:
score = 85

if score >= 50:                                  # Level 0 — no indent
    print("You passed.")                        # Level 1 — 4 spaces
    if score >= 90:                             # Level 1 — still inside first
if
        print("Excellent performance!")         # Level 2 — 8 spaces

print("Grading complete.")                      # Level 0 — outside both if
blocks
```

Figure 2.9 — Nested indentation levels

Chapter 3

Variables and Data Types

Every program works with data — names, numbers, dates, lists, and more. In this chapter, you will learn how Python stores and manages different kinds of data using variables and data types. These are the fundamental building blocks of any Python program.

3.1 Variables in Python

When a program processes data, it needs a way to store that data temporarily in memory so that it can be used later. This is where variables come in.

Variable	A named storage location in the computer's memory that holds a value. You give the variable a name, and you can use that name anywhere in your program to access or change the stored value.

🔎 Real-World Analogy

Think of a variable like a labelled box.

You have a box, and you write 'Student Name' on its label.

Inside the box, you place the paper that says 'Alice'.

Whenever you need to know the student's name, you go to the box labelled 'Student Name' and read its contents.

If the student's name changes, you replace the paper inside the box — but the label (variable name) stays the same.

Creating a Variable — Assignment

In Python, you create a variable simply by giving it a name and assigning it a value using the equals sign (=). The equals sign in programming is called the assignment operator — it means "store this value in this variable".

```
# Creating variables and assigning values:
student_name = "Alice"        # Stores the text "Alice"
student_age  = 21             # Stores the number 21
student_gpa  = 8.5            # Stores the decimal 8.5
is_enrolled  = True           # Stores True or False

# Displaying variable values:
print(student_name)
print(student_age)
print(student_gpa)
print(is_enrolled)
```

Figure 3.1 — Creating and using variables

Output

```
Alice
21
8.5
True
```

Changing a Variable's Value

A variable's value can be updated at any point in the program. Python will always use the most recent value assigned to a variable.

```
# Updating a variable:
price = 500
print("Original price:", price)

price = 450              # Updated after a discount
print("Discounted price:", price)

price = price - 50       # Updated again using its own value
print("Final price:", price)
```

Figure 3.2 — Updating variable values

Output

Original price: 500

Discounted price: 450

Final price: 400

3.2 Variable Naming Rules

Not every sequence of characters can be used as a variable name. Python has specific rules that must be followed, as well as conventions (best practices) that are strongly recommended.

Mandatory Rules — Must Follow

Breaking these rules will result in a SyntaxError and your program will not run:

1. **Must start with a letter or underscore (_)** — Cannot start with a number.
2. **Can only contain letters, numbers, and underscores** — No spaces, hyphens, or special characters.
3. **Case-sensitive** — student_name and Student_Name are two different variables.
4. **Cannot be a Python keyword** — Reserved words like if, for, while, True, False, None cannot be used as variable names.

Valid Variable Names	Invalid Variable Names	Why Invalid
name	1name	Starts with a number
student_age	student age	Contains a space
_total	total-cost	Contains a hyphen (-)
courseCode2025	if	Reserved Python keyword
firstName	first.name	Contains a period (.)

Naming Conventions — Best Practices

These are not enforced by Python, but are widely followed by professional developers to keep code consistent and readable:

- **Use snake_case for variable names:** Write multi-word names with underscores between words. Example: `student_name`, `total_price`, `is_active`
- **Use meaningful names:** Variable names should describe what they hold. Avoid names like `x` or `abc` unless in a specific mathematical context.
- **Avoid single-character names:** Unless used in short loops (e.g., `i` for index), prefer descriptive names.

```
# Poor naming — hard to understand:
a = "Alice"
b = 21
c = 8.5

# Good naming — immediately clear:
student_name = "Alice"
student_age  = 21
student_gpa  = 8.5
```

Figure 3.3 — Good vs. poor variable naming

3.3 Basic Data Types in Python

Every value stored in a variable has a type. The type tells Python how much memory to reserve and what operations are allowed on that value. Python automatically figures out the type based on the value you assign — you do not need to declare the type explicitly (this is called dynamic typing).

Integer — int

int	Represents whole numbers — both positive and negative — without any decimal point. Examples: 0, 42, -100, 2025

```
# Integer examples:
year        = 2025
temperature = -5
student_count = 120
floors_in_building = 30

print(type(year))           # <class "int">
print(type(temperature))    # <class "int">

# Arithmetic with integers:
total_marks = 450
```

```
subjects    = 5
average     = total_marks // subjects    # Integer division
print("Average marks:", average)         # Output: 90
```

Figure 3.4 — Integer data type

Float — float

float	Represents real numbers with a decimal point. Used whenever precision beyond whole numbers is needed. Examples: 3.14, -0.5, 99.99, 2.0

```
# Float examples:
price   = 1499.99
weight  = 72.5
gpa     = 8.75
pi      = 3.14159

print(type(price))    # <class "float">

# Calculation with floats:
product_price   = 850.00
discount_rate   = 0.15          # 15% discount
discount_amount = product_price * discount_rate
final_price     = product_price - discount_amount

print("Final price after discount:", final_price)
# Output: Final price after discount: 722.5
```

Figure 3.5 — Float data type

String — str

str	Represents text — any sequence of characters enclosed in single quotes (' ') or double quotes (" "). Examples: "Hello", 'Alice', "2025-01-15"

```
# String examples:
first_name  = "Alice"
last_name   = 'Sharma'
email       = "alice.sharma@example.com"
address     = "204, Tech Park, Ahmedabad"

print(type(first_name))    # <class "str">

# Combining strings (called concatenation):
full_name = first_name + " " + last_name
print("Full name:", full_name)
```

```
# Output: Full name: Alice Sharma

# String with numbers (the number becomes text inside quotes):
employee_id = "EMP-2025-001"
print(employee_id)
# Output: EMP-2025-001
```

Figure 3.6 — String data type

Pro Tip

Numbers stored as strings cannot be used in calculations directly. "25" + "10" gives "2510" (text joined together), not 35. To add them mathematically, convert to int first: int("25") + int("10") = 35.

Boolean — bool

bool	Represents one of only two possible values: True or False. Used in decision-making and conditions. Note: True and False must be written with a capital first letter in Python.

```
# Boolean examples:
is_logged_in     = True
is_admin         = False
has_paid         = True
account_active   = False

print(type(is_logged_in))    # <class "bool">

# Booleans from comparisons:
age = 20
print(age >= 18)     # Output: True  (20 is >= 18)
print(age > 25)      # Output: False (20 is not > 25)

# Using booleans in a condition:
if has_paid:
    print("Access granted. Thank you for your payment.")
else:
    print("Please complete your payment to continue.")
```

Figure 3.7 — Boolean data type

None — NoneType

None	A special value in Python that represents the absence of a value. It is used when a variable has been declared but no value has been assigned to it yet, or when a function intentionally returns nothing.

```
# None examples:
result          = None    # No result yet — still processing
selected_option = None    # User has not made a selection

print(result)             # Output: None
print(type(result))       # <class "NoneType">

# Checking for None:
if result is None:
    print("The calculation has not been performed yet.")
else:
    print("Result:", result)
```

Figure 3.8 — NoneType data type

List — list

list	An ordered, changeable collection of items. A list can hold multiple values — even of different types — inside square brackets [ ]. Items are separated by commas.

```
# List examples:
subjects       = ["Mathematics", "Physics", "Chemistry", "English"]
marks          = [88, 92, 76, 95]
employee_ids   = [1001, 1002, 1003, 1004]
mixed_list     = ["Alice", 21, True, 8.5]    # Multiple types in one list

# Accessing items by position (index starts at 0):
print(subjects[0])    # Output: Mathematics
print(subjects[1])    # Output: Physics
print(subjects[-1])   # Output: English  (last item)

# Adding an item to the list:
subjects.append("Computer Science")
print(subjects)
# Output: ["Mathematics", "Physics", "Chemistry", "English", "Computer
Science"]

# Finding the number of items:
print(len(subjects))    # Output: 5
```

Figure 3.9 — List data type

🔎 List Index — How Positions Work

Python counts positions starting from 0, not 1. This is called zero-based indexing.

List: ["Mathematics", "Physics", "Chemistry", "English"]

Index: 0 1 2 3

To access the first item, use list[0]. To access the last, use list[-1].

Tuple — tuple

tuple	An ordered, unchangeable (immutable) collection of items, enclosed in parentheses ( ). Once created, the values in a tuple cannot be added, removed, or changed.

```
# Tuple examples:
coordinates     = (28.6139, 77.2090)      # Latitude and Longitude of Delhi
rgb_red         = (255, 0, 0)             # Color code for red
days_of_week    = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")

# Accessing items:
print(coordinates[0])      # Output: 28.6139
print(days_of_week[4])     # Output: Fri

# Trying to change a tuple causes an error:
# coordinates[0] = 30.0   # TypeError: 'tuple' object does not support item
assignment

# Finding the length:
print(len(days_of_week))   # Output: 7
```

Figure 3.10 — Tuple data type

Note

Use a tuple when you have data that should never change — such as GPS coordinates, RGB color codes, or the days of the week. Use a list when the data may need to be modified.

Dictionary — dict

dict	A collection of key-value pairs, enclosed in curly braces { }. Each item has a unique key and an associated value. Dictionaries are used when data needs to be looked up by a name (key) rather than a position.

```
# Dictionary example — student record:
student = {
```

```
    "name"    : "Alice Sharma",
    "age"     : 21,
    "course"  : "B.Tech Computer Science",
    "gpa"     : 8.75,
    "active"  : True
}

# Accessing values by key:
print(student["name"])       # Output: Alice Sharma
print(student["gpa"])        # Output: 8.75

# Adding a new key-value pair:
student["graduation_year"] = 2026

# Updating an existing value:
student["gpa"] = 9.0

# Displaying all keys and values:
for key, value in student.items():
    print(key, ":", value)
```

Figure 3.11 — Dictionary data type

Output

Alice Sharma

8.75

name : Alice Sharma

age : 21

course : B.Tech Computer Science

gpa : 9.0

active : True

graduation_year : 2026

Set — set

set	An unordered collection of unique items, enclosed in curly braces { }. Sets automatically remove duplicate values. They are useful for membership testing and mathematical operations like union and intersection.

```
# Set example:
```

```
registered_courses = {"Python", "Data Science", "Machine Learning",
"Python"}
# Note: "Python" appears twice, but a set removes duplicates automatically

print(registered_courses)
# Output: {"Python", "Data Science", "Machine Learning"}  (order may vary)

# Adding an item:
registered_courses.add("Web Development")

# Checking membership:
print("Python" in registered_courses)       # Output: True
print("Java" in registered_courses)         # Output: False

# Set operations:
morning_batch   = {"Alice", "Bob", "Charlie"}
evening_batch   = {"Charlie", "David", "Eve"}

# Students in both batches (intersection):
both_batches    = morning_batch & evening_batch
print(both_batches)    # Output: {"Charlie"}

# All unique students (union):
all_students    = morning_batch | evening_batch
print(all_students)    # Output: {"Alice", "Bob", "Charlie", "David", "Eve"}
```


Figure 3.12 — Set data type

3.4 Data Types — Quick Reference Summary

The following table summarizes all the core data types covered in this chapter.

Data Type	Keyword	Example	Mutable?	Ordered?
Integer	int	age = 21	Yes	N/A
Float	float	price = 99.99	Yes	N/A
String	str	"Alice"	No	Yes
Boolean	bool	is_active = True	Yes	N/A
None	None	result = None	N/A	N/A
List	list	["Math", "Science", "English"]	Yes	Yes
Tuple	tuple	("Mon", "Tue", "Wed")	No	Yes
Dictionary	dict	{"name": "Alice", "age": 21}	Yes	Yes (3.7+)

Set	set	{"Python", "Data Science"}	Yes	No

Checking the Type of a Variable

Python provides a built-in function called type() that tells you the data type of any value or variable.

```
# Using type() to check data types:
print(type(42))           # <class "int">
print(type(3.14))         # <class "float">
print(type("Hello"))      # <class "str">
print(type(True))         # <class "bool">
print(type(None))         # <class "NoneType">
print(type([1, 2, 3]))    # <class "list">
print(type((1, 2, 3)))    # <class "tuple">
print(type({"a": 1}))     # <class "dict">
print(type({1, 2, 3}))    # <class "set">
```

Figure 3.13 — Using type() to inspect data types

Chapter Summary

In this chapter, you have covered the essential foundations of Python programming. Here is a recap of the key concepts:

- **Python** is a high-level, interpreted, general-purpose programming language known for its readable syntax and versatility.
- **Guido van Rossum** created Python in 1989, and it has since grown into one of the world's most popular languages.
- Python is used in Data Science, AI, Web Development, and Automation across major industries.
- **Syntax** defines the rules of Python code — it must be written precisely for the program to execute.
- **Comments** (using #) make code more readable and maintainable.
- **Indentation** (4 spaces) is mandatory in Python and defines code blocks.
- **Variables** store values in named memory locations. They are created using the assignment operator (=).
- Python has nine fundamental data types: int, float, str, bool, None, list, tuple, dict, and set — each suited to different kinds of data.